



# Sesi 4: Model & Data Persistence



## Overview Sesi

**Durasi:** 90 menit (1.5 jam) | **Branch:** 08-task-model → 09-shared-preferences → 10-persistent-data | **Tujuan:** Belajar membuat Model class dan menyimpan data secara persistent dengan SharedPreferences

---



## Agenda Sesi

### Bagian 1: Create Task Model (30 menit)

- ✓ Generate Model dengan Metro CLI
- ✓ Understand data serialization (toJson, fromJson)
- ✓ Implement Task model class
- ✓ Convert Map to Model object

### Bagian 2: SharedPreferences Setup (30 menit)

- ✓ Install shared\_preferences package
- ✓ Initialize SharedPreferences
- ✓ Save dan load data
- ✓ Handle JSON encoding/decoding

### Bagian 3: Persistent CRUD (30 menit)

- ✓ Save tasks to storage after create
  - ✓ Load tasks from storage on app start
  - ✓ Update storage on edit/delete
  - ✓ Clear storage functionality
- 



## Bagian 1: Create Task Model (30 menit)

### 1.1 What is a Model?

**Model** adalah class yang merepresentasikan struktur data. Seperti blueprint atau template.

**Analogi:** - **Map<String, dynamic>** = Kardus tanpa label (bisa isi apa aja, tapi berantakan) - **Model Class** = Kotak dengan label jelas (isinya terstruktur dan rapi)

**Keuntungan pakai Model:** -  **Type safety:** Dart tahu tipe data setiap property - 

**Autocomplete:** IDE bisa suggest property names -  **Reusable:** Pakai di banyak tempat dengan struktur sama -  **Validation:** Bisa tambah validasi di constructor - 

**Methods:** Bisa tambah helper methods

---

## 1.2 Generate Task Model

```
dart run nylo_framework:main make:model task
```

**Output:**

✓ Created lib/app/models/task.dart

---

## 1.3 Implement Task Model

**Edit file:** lib/app/models/task.dart

```
class Task {
  // Properties
  String id;
  String title;
  String description;
  bool isCompleted;
  DateTime createdAt;

  // Constructor
  Task({
    required this.id,
    required this.title,
    this.description = '',
    this.isCompleted = false,
    required this.createdAt,
  });

  // Convert dari JSON (Map) ke Task object
  // Dipakai saat load data dari storage
  factory Task.fromJson(Map<String, dynamic> json) {
    return Task(
      id: json['id'] as String,
      title: json['title'] as String,
      description: json['description'] as String? ?? '',
      isCompleted: json['isCompleted'] as bool? ?? false,
      createdAt: DateTime.parse(json['createdAt'] as String),
    );
  }

  // Convert dari Task object ke JSON (Map)
  // Dipakai saat save data ke storage
  Map<String, dynamic> toJson() {
    return {
      'id': id,
```

```

        'title': title,
        'description': description,
        'isCompleted': isCompleted,
        'createdAt': createdAt.toIso8601String(),
    };
}

// Helper method untuk copy dengan perubahan
Task copyWith({
    String? id,
    String? title,
    String? description,
    bool? isCompleted,
    DateTime? createdAt,
}) {
    return Task(
        id: id ?? this.id,
        title: title ?? this.title,
        description: description ?? this.description,
        isCompleted: isCompleted ?? this.isCompleted,
        createdAt: createdAt ?? this.createdAt,
    );
}

// Override toString untuk debugging
@override
String toString() {
    return 'Task(id: $id, title: $title, isCompleted: $isCompleted)';
}
}

```

### 💡 Penjelasan:

#### 1. Constructor:

```

Task({
    required this.id,          // Wajib diisi
    required this.title,       // Wajib diisi
    this.description = '',     // Optional, default empty string
    this.isCompleted = false,  // Optional, default false
    required this.createdAt,   // Wajib diisi
})

```

**2. fromJson() - Factory Constructor:** - factory = Constructor special yang bisa return object - Dipakai untuk convert Map<String, dynamic> → Task - Pakai as String untuk type casting dengan aman - Pakai ?? untuk default value jika null

**3. toJson() - Regular Method:** - Return Map<String, dynamic> - Dipakai untuk convert Task → Map (untuk save ke storage) - DateTime di-convert ke ISO8601 string format

**4. copyWith() - Helper Method:** - Buat copy object dengan beberapa property berubah - Useful untuk immutable updates

---

## 1.4 Update Controller untuk Pakai Model

Edit file: `lib/app/controllers/todo_home_controller.dart`

Import Task model:

```
import 'package:flutter/material.dart';
import 'package:nylo_framework/nylo_framework.dart';
import '/app/models/task.dart'; // Import Task model
```

Ubah state dari `List<Map>` ke `List<Task>`:

```
class TodoHomeController extends Controller {
  // State: List tasks dengan Task model (bukan Map lagi!)
  List<Task> tasks = [];

  @override
  construct(BuildContext context) {
    super.construct(context);

    // Initialize with sample data using Task model
    tasks = [
      Task(
        id: '1',
        title: 'Belajar Flutter',
        description: 'Selesaikan tutorial Flutter basics',
        isCompleted: false,
        createdAt: DateTime.now(),
      ),
      Task(
        id: '2',
        title: 'Setup Project Nylo',
        description: 'Install dan setup Nylo framework',
        isCompleted: true,
        createdAt: DateTime.now(),
      ),
      Task(
        id: '3',
        title: 'Build ToDo App',
        description: 'Buat aplikasi ToDo sederhana',
        isCompleted: false,
        createdAt: DateTime.now(),
      ),
    ];
  }

  // Helper getters - masih sama
  int get totalTasks => tasks.length;

  int get completedTasks => tasks.where((t) => t.isCompleted).length;

  int get pendingTasks => tasks.where((t) => !t.isCompleted).length;

  // Update addTask method
  void addTask({
```

```

        required String title,
        String description = '',
    }) {
        String newId = DateTime.now().millisecondsSinceEpoch.toString();

        // Create Task object (bukan Map lagi!)
        Task newTask = Task(
            id: newId,
            title: title,
            description: description,
            isCompleted: false,
            createdAt: DateTime.now(),
        );

        tasks.insert(0, newTask);
        setState(() {});

        print('Task added: $newTask');
    }

    // Update updateTask method
    void updateTask({
        required String id,
        String? title,
        String? description,
        bool? isCompleted,
    }) {
        final taskIndex = tasks.indexWhere((t) => t.id == id);

        if (taskIndex == -1) {
            print('Task not found: $id');
            return;
        }

        // Pakai copyWith untuk update
        tasks[taskIndex] = tasks[taskIndex].copyWith(
            title: title,
            description: description,
            isCompleted: isCompleted,
        );

        setState(() {});
        print('Task updated: ${tasks[taskIndex]}');
    }

    // Update toggleComplete method
    void toggleComplete(String id) {
        final taskIndex = tasks.indexWhere((t) => t.id == id);

        if (taskIndex != -1) {
            tasks[taskIndex] = tasks[taskIndex].copyWith(
                isCompleted: !tasks[taskIndex].isCompleted,
            );
            setState(() {});
        }
    }

```

```

}

// Update deleteTask method - tetap sama
void deleteTask(String id) {
  tasks.removeWhere((t) => t.id == id);
  setState(() {});
  print('Task deleted: $id');
}

// Filter methods
List<Task> getTasksByStatus(bool isCompleted) {
  return tasks.where((t) => t.isCompleted == isCompleted).toList();
}

List<Task> get pendingTasksList => getTasksByStatus(false);
List<Task> get completedTasksList => getTasksByStatus(true);
}

```

---

## 1.5 Update Pages untuk Pakai Task Model

Edit file: `lib/resources/pages/todo_home_page.dart`

Update `_buildTaskList()` - ganti type hint:

```

Widget _buildTaskList() {
  final controller = widget.controller;

  // Sekarang List<Task>, bukan List<Map> lagi!
  List<Task> tasks = controller.tasks;

  if (tasks.isEmpty) {
    return Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Icon(Icons.inbox, size: 80, color: Colors.grey.shade300),
          SizedBox(height: 16),
          Text(
            'No tasks yet!',
            style: TextStyle(
              fontSize: 20,
              color: Colors.grey.shade600,
              fontWeight: FontWeight.bold,
            ),
          ),
          SizedBox(height: 8),
          Text(
            'Tap the + button to add your first task',
            style: TextStyle(
              fontSize: 14,
              color: Colors.grey.shade500,
            ),
          ),
        ],
      ),
    );
  }
}

```

```

    ),
  );
}

return ListView.builder(
  padding: EdgeInsets.all(16),
  itemCount: tasks.length,
  itemBuilder: (context, index) {
    final task = tasks[index];
    return _buildTaskCard(task);
  },
);
}

```

**Update \_buildTaskCard() - parameter type dari Map ke Task:**

```

Widget _buildTaskCard(Task task) { // Ubah dari Map ke Task
  bool isCompleted = task.isCompleted; // Akses property langsung

  return Card(
    elevation: 2,
    margin: EdgeInsets.only(bottom: 12),
    child: ListTile(
      leading: CircleAvatar(
        backgroundColor: isCompleted ? Colors.green : Colors.orange,
        child: Icon(
          isCompleted ? Icons.check : Icons.circle_outlined,
          color: Colors.white,
        ),
      ),
      title: Text(
        task.title, // Akses property langsung, bukan task['title']
        style: TextStyle(
          fontSize: 16,
          fontWeight: FontWeight.bold,
          decoration: isCompleted ? TextDecoration.lineThrough : null,
        ),
      ),
      subtitle: Text(
        task.description, // Akses property langsung
        style: TextStyle(fontSize: 14, color: Colors.grey),
      ),
      trailing: Icon(Icons.arrow_forward_ios, size: 16),
      onTap: () {
        routeTo(
          '/detail-task',
          data: task.toJson(), // Convert ke Map untuk passing data
        );
      },
    ),
  );
}

```

**Edit file: lib/resources/pages/detail\_task\_page.dart**

**Update init() method untuk convert Map ke Task:**

```

import '/app/models/task.dart'; // Import Task model

class _DetailTaskPageState extends NyState<DetailTaskPage> {
  Task? task; // Ubah dari Map ke Task

  @override
  init() async {
    super.init();

    // Ambil data dari route arguments
    final taskData = widget.data() as Map<String, dynamic>;

    if (taskData != null) {
      // Convert Map ke Task object
      task = Task.fromJson(taskData);
    }
  }

  // Rest of the code...
}

```

**Update method yang akses task properties:**

```

@override
Widget view(BuildContext context) {
  if (task == null) {
    return Scaffold(
      appBar: AppBar(title: Text('Error')),
      body: Center(child: Text('Task not found')),
    );
  }

  bool isCompleted = task!.isCompleted; // Akses property langsung

  return Scaffold(
    appBar: AppBar(
      title: Text('Task Detail'),
      backgroundColor: Colors.blue,
      foregroundColor: Colors.white,
      actions: [
        IconButton(
          icon: Icon(Icons.edit),
          onPressed: () {
            print('Edit button diklik');
          },
        ),
        IconButton(
          icon: Icon(Icons.delete),
          onPressed: () {
            _showDeleteConfirmation();
          },
        ),
      ],
    ),
    body: SingleChildScrollView(

```

```

padding: EdgeInsets.all(16),
child: Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    // Status badge
    Container(
      padding: EdgeInsets.symmetric(horizontal: 12, vertical:
6),
      decoration: BoxDecoration(
        color: isCompleted ? Colors.green : Colors.orange,
        borderRadius: BorderRadius.circular(20),
      ),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          Icon(
            isCompleted ? Icons.check_circle : Icons.pending,
            color: Colors.white,
            size: 16,
          ),
          SizedBox(width: 4),
          Text(
            isCompleted ? 'Completed' : 'Pending',
            style: TextStyle(color: Colors.white),
          ),
        ],
      ),
    ),
    SizedBox(height: 20),

    // Title
    Text(
      'Title',
      style: TextStyle(
        fontSize: 12,
        color: Colors.grey,
        fontWeight: FontWeight.bold,
      ),
    ),
    SizedBox(height: 4),
    Text(
      task!.title, // Akses property langsung
      style: TextStyle(
        fontSize: 24,
        fontWeight: FontWeight.bold,
      ),
    ),
    SizedBox(height: 20),

    // Description
    Text(
      'Description',
      style: TextStyle(
        fontSize: 12,
        color: Colors.grey,

```

```

        fontWeight: FontWeight.bold,
      ),
    ),
    SizedBox(height: 4),
    Text(
      task!.description.isEmpty ? 'No description' :
task!.description,
      style: TextStyle(fontSize: 16),
    ),
    SizedBox(height: 32),

    // Toggle complete button
    SizedBox(
      width: double.infinity,
      height: 50,
      child: ElevatedButton.icon(
        onPressed: () {
          final homeController =
nyNavigator.navigator.getControllerOfType<TodoHomeController>
();

          if (homeController != null && task != null) {
            homeController.toggleComplete(task!.id);

            setState(() {
              task = task!.copyWith(isCompleted: !isCompleted);
            });

            showToastNotification(
              context,
              title: 'Success',
              description: isCompleted
                ? 'Task marked as pending'
                : 'Task marked as completed',
              style: ToastNotificationStyleType.SUCCESS,
            );
          }
        },
        icon: Icon(isCompleted ? Icons.undo : Icons.check),
        label: Text(
          isCompleted ? 'Mark as Pending' : 'Mark as Completed',
          style: TextStyle(fontSize: 16),
        ),
        style: ElevatedButton.styleFrom(
          backgroundColor: isCompleted ? Colors.orange :
Colors.green,
          foregroundColor: Colors.white,
        ),
      ),
    ),
  ],
),
),
);
}

```

```

void _handleDelete() {
  final homeController =
    nyNavigator.navigator.getControllerOfType<TodoHomeController>
    ();

  if (homeController != null && task != null) {
    homeController.deleteTask(task!.id);

    showToastNotification(
      context,
      title: 'Deleted',
      description: 'Task has been deleted',
      style: ToastNotificationStyleType.WARNING,
    );

    Future.delayed(Duration(milliseconds: 500), () {
      pop(context);
    });
  }
}

```

🔥 Hot reload dan test! App masih berfungsi normal, tapi sekarang pakai Model class!

---

## Bagian 2: SharedPreferences Setup (30 menit)

### 2.1 What is SharedPreferences?

**SharedPreferences** adalah storage lokal untuk menyimpan data simple seperti key-value pairs.

**Analogi:** - **In-Memory (List)** = Kertas catatan (hilang kalau app ditutup) -  
**SharedPreferences** = Buku catatan (tetap ada meski app ditutup)

**Kapan pakai SharedPreferences?** - ☒ Data simple (settings, user preferences, small data) - ☒ Data yang perlu persist setelah app ditutup - ☒ Quick access (tidak perlu koneksi internet)

**Kapan TIDAK pakai SharedPreferences?** - ☒ Data besar (gambar, video, file) - ☒  
 Data sensitive (password, credit card) - pakai secure storage - ☒ Data relational (banyak tabel dengan relasi) - pakai database

---

### 2.2 Install shared\_preferences Package

Edit file: pubspec.yaml

```

dependencies:
  flutter:
    sdk: flutter

```

```
nylo_framework: ^6.9.1
shared_preferences: ^2.5.3 # Tambahkan ini
```

### Install package:

```
flutter pub get
```

---

## 2.3 Create Storage Service

Mari buat service class untuk handle semua operasi storage.

Create file: `lib/app/services/storage_service.dart`

```
import 'dart:convert';
import 'package:shared_preferences/shared_preferences.dart';
import '/app/models/task.dart';

class StorageService {
  // Storage key
  static const String tasksKey = 'tasks';

  // Get SharedPreferences instance
  Future<SharedPreferences> get _prefs async {
    return await SharedPreferences.getInstance();
  }

  /// Save tasks to storage
  Future<bool> saveTasks(List<Task> tasks) async {
    try {
      final prefs = await _prefs;

      // Convert List<Task> ke List<Map>
      List<Map<String, dynamic>> tasksJson =
        tasks.map((task) => task.toJson()).toList();

      // Convert List<Map> ke JSON string
      String tasksString = jsonEncode(tasksJson);

      // Save ke SharedPreferences
      bool result = await prefs.setString(tasksKey, tasksString);

      print('✅ Saved ${tasks.length} tasks to storage');
      return result;
    } catch (e) {
      print('❌ Error saving tasks: $e');
      return false;
    }
  }

  /// Load tasks from storage
  Future<List<Task>> loadTasks() async {
    try {
      final prefs = await _prefs;
```

```

// Get JSON string dari SharedPreferences
String? tasksString = prefs.getString(tasksKey);

// Jika null, return empty list
if (tasksString == null || tasksString.isEmpty) {
    print('❗ No tasks in storage');
    return [];
}

// Convert JSON string ke List<dynamic>
List<dynamic> tasksJson = jsonDecode(tasksString);

// Convert List<dynamic> ke List<Task>
List<Task> tasks = tasksJson
    .map((json) => Task.fromJson(json as Map<String, dynamic>))
    .toList();

print('✅ Loaded ${tasks.length} tasks from storage');
return tasks;
} catch (e) {
    print('❌ Error loading tasks: $e');
    return [];
}
}

/// Clear all tasks from storage
Future<bool> clearTasks() async {
    try {
        final prefs = await _prefs;
        bool result = await prefs.remove(tasksKey);

        print('✅ Cleared tasks from storage');
        return result;
    } catch (e) {
        print('❌ Error clearing tasks: $e');
        return false;
    }
}

/// Check if tasks exist in storage
Future<bool> hasTasks() async {
    final prefs = await _prefs;
    return prefs.containsKey(tasksKey);
}
}

```

### 💡 Penjelasan flow:

**Save:** 1. List<Task> → List<Map> (pakai toJson()) 2. List<Map> → String (pakai toJson()) 3. Save string ke SharedPreferences

**Load:** 1. Get string dari SharedPreferences 2. String → List<dynamic> (pakai jsonDecode()) 3. List<dynamic> → List<Task> (pakai fromJson())

---



## Bagian 3: Persistent CRUD (30 menit)

### 3.1 Update Controller dengan Storage

Edit file: `lib/app/controllers/todo_home_controller.dart`

Import storage service:

```
import 'package:flutter/material.dart';
import 'package:nylo_framework/nylo_framework.dart';
import '/app/models/task.dart';
import '/app/services/storage_service.dart'; // Import storage
```

Add storage instance:

```
class TodoHomeController extends Controller {
  // State
  List<Task> tasks = [];
  bool isLoading = false;

  // Storage service
  final StorageService storage = StorageService();

  @override
  construct(BuildContext context) {
    super.construct(context);

    // Load tasks from storage saat pertama kali
    loadTasksFromStorage();
  }

  // Load tasks from storage
  Future<void> loadTasksFromStorage() async {
    setState(() {
      isLoading = true;
    });

    try {
      // Load dari storage
      List<Task> loadedTasks = await storage.loadTasks();

      // Jika storage kosong, pakai sample data
      if (loadedTasks.isEmpty) {
        tasks = [
          Task(
            id: '1',
            title: 'Belajar Flutter',
            description: 'Selesaikan tutorial Flutter basics',
            isCompleted: false,
            createdAt: DateTime.now(),
          ),
          Task(
            id: '2',
            title: 'Setup Project Nylo',

```

```

        description: 'Install dan setup Nylo framework',
        isCompleted: true,
        createdAt: DateTime.now(),
    ),
    Task(
        id: '3',
        title: 'Build ToDo App',
        description: 'Buat aplikasi ToDo sederhana',
        isCompleted: false,
        createdAt: DateTime.now(),
    ),
];

// Save sample data ke storage
await storage.saveTasks(tasks);
} else {
    tasks = loadedTasks;
}

setState(() {
    isLoading = false;
});
} catch (e) {
    print('Error loading tasks: $e');
    setState(() {
        isLoading = false;
    });
}
}

// Save tasks to storage
Future<void> saveTasksToStorage() async {
    await storage.saveTasks(tasks);
}

// Getters - tetap sama
int get totalTasks => tasks.length;
int get completedTasks => tasks.where((t) => t.isCompleted).length;
int get pendingTasks => tasks.where((t) => !t.isCompleted).length;

// Update addTask untuk save ke storage
Future<void> addTask({
    required String title,
    String description = '',
}) async {
    String newId = DateTime.now().millisecondsSinceEpoch.toString();

    Task newTask = Task(
        id: newId,
        title: title,
        description: description,
        isCompleted: false,
        createdAt: DateTime.now(),
    );

```

```

tasks.insert(0, newTask);

// Save ke storage
await saveTasksToStorage();

setState(() {});
print('Task added and saved: $newTask');
}

// Update updateTask untuk save ke storage
Future<void> updateTask({
  required String id,
  String? title,
  String? description,
  bool? isCompleted,
}) async {
  final taskIndex = tasks.indexWhere((t) => t.id == id);

  if (taskIndex == -1) {
    print('Task not found: $id');
    return;
  }

  tasks[taskIndex] = tasks[taskIndex].copyWith(
    title: title,
    description: description,
    isCompleted: isCompleted,
  );

  // Save ke storage
  await saveTasksToStorage();

  setState(() {});
  print('Task updated and saved: ${tasks[taskIndex]}');
}

// Update toggleComplete untuk save ke storage
Future<void> toggleComplete(String id) async {
  final taskIndex = tasks.indexWhere((t) => t.id == id);

  if (taskIndex != -1) {
    tasks[taskIndex] = tasks[taskIndex].copyWith(
      isCompleted: !tasks[taskIndex].isCompleted,
    );

    // Save ke storage
    await saveTasksToStorage();

    setState(() {});
  }
}

// Update deleteTask untuk save ke storage
Future<void> deleteTask(String id) async {
  tasks.removeWhere((t) => t.id == id);

```

```

    // Save ke storage
    await saveTasksToStorage();

    setState(() {});
    print('Task deleted and saved');
  }

  // Clear all tasks
  Future<void> clearAllTasks() async {
    tasks.clear();
    await storage.clearTasks();
    setState(() {});
    print('All tasks cleared');
  }

  // Filter methods - tetap sama
  List<Task> getTasksByStatus(bool isCompleted) {
    return tasks.where((t) => t.isCompleted == isCompleted).toList();
  }

  List<Task> get pendingTasksList => getTasksByStatus(false);
  List<Task> get completedTasksList => getTasksByStatus(true);
}

```

---

### 3.2 Update AddTaskPage

Edit file: `lib/resources/pages/add_task_page.dart`

Ubah `_handleSave()` jadi `async`:

```

Future<void> _handleSave() async {
  String title = titleController.text.trim();
  String description = descriptionController.text.trim();

  if (title.isEmpty) {
    showToastNotification(
      context,
      title: 'Error',
      description: 'Task title tidak boleh kosong!',
      style: ToastNotificationStyleType.DANGER,
    );
    return;
  }

  final homeController =
    navigator.navigator.getControllerOfType<TodoHomeController>
    ();

  if (homeController != null) {
    // Call addTask method (sekarang async)
    await homeController.addTask(
      title: title,
      description: description,
    );
  }
}

```

```

);

showToastNotification(
  context,
  title: 'Success',
  description: 'Task berhasil ditambahkan dan disimpan!',
  style: ToastNotificationStyleType.SUCCESS,
);

Future.delayed(Duration(seconds: 1), () {
  pop(context);
});
}
}

```

---

### 3.3 Test Persistence

**Test flow:**

1. Add beberapa tasks baru
2. Close app completely (stop dari terminal atau kill app)
3. Run app lagi
4. Tasks masih ada! 🎉

**Test Checklist:**

- ☐ Add task → Close app → Open app → Task masih ada
  - ☐ Toggle complete → Close app → Open app → Status tetap
  - ☐ Delete task → Close app → Open app → Task sudah hilang
  - ☐ Edit task → Close app → Open app → Perubahan tersimpan
- 

### 🎨 Bonus: Clear All Tasks

Tambahkan button untuk clear semua tasks (useful untuk testing).

**Edit file: lib/resources/pages/todo\_home\_page.dart**

**Tambahkan menu di AppBar:**

**@override**

```

Widget view(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text('My Todo List'),
      backgroundColor: Colors.blue,
      foregroundColor: Colors.white,
      actions: [
        PopupMenuButton<String>(
          onSelected: (value) {
            if (value == 'clear') {
              _showClearConfirmation();
            }
          }
        )
      ]
    )
  );
}

```

```

    },
    itemBuilder: (context) => [
      PopupMenuItem(
        value: 'clear',
        child: Row(
          children: [
            Icon(Icons.delete_sweep, color: Colors.red),
            SizedBox(width: 8),
            Text('Clear All Tasks'),
          ],
        ),
      ),
    ],
  ),
  ],
),
body: Column(
  children: [
    _buildStatsCard(),
    Expanded(child: _buildTaskList()),
  ],
),
floatingActionButton: FloatingActionButton(
  onPressed: () {
    routeTo('/add-task');
  },
  child: Icon(Icons.add),
),
);
}

void _showClearConfirmation() {
  showDialog(
    context: context,
    builder: (context) => AlertDialog(
      title: Text('Clear All Tasks?'),
      content: Text('This will delete all tasks permanently. Are you sure?'),
      actions: [
        TextButton(
          onPressed: () {
            pop(context);
          },
          child: Text('Cancel'),
        ),
        TextButton(
          onPressed: () async {
            final controller = widget.controller;
            await controller.clearAllTasks();

            pop(context);

            showToastNotification(
              context,
              title: 'Cleared',

```

```

        description: 'All tasks have been deleted',
        style: ToastNotificationStyle.WARNING,
    );
},
style: TextButton.styleFrom(foregroundColor: Colors.red),
child: Text('Clear All'),
),
1,
),
);
}

```

---

## Common Issues & Solutions

### Issue 1: “Tasks not loading on app start”

**Symptom:** App starts with empty list setiap kali **Solution:** - Check if `loadTasksFromStorage()` dipanggil di `construct()` - Verify `tasksKey` string sama saat save dan load - Check console untuk error messages

### Issue 2: “FormatException: Unexpected character”

**Symptom:** Error saat decode JSON **Solution:** - Data corrupted di storage, clear dengan `storage.clearTasks()` - Verify `toJson()` return valid JSON structure

```
await storage.clearTasks(); // Clear corrupted data
```

### Issue 3: “Tasks duplicate after reload”

**Symptom:** Tasks muncul double setelah restart **Solution:** - Jangan init dengan sample data jika storage sudah ada data - Check if `loadedTasks.isEmpty` sebelum add sample data

### Issue 4: “Storage not saving”

**Symptom:** Changes tidak persist **Solution:** - Verify semua CRUD methods call `saveTasksToStorage()` - Check if `async/await` properly implemented - Test with simple debug print di `saveTasks()` method

---

## Key Takeaways

✅ **Model Class:** - Struktur data terorganisir dengan type safety - `fromJson()` untuk deserialize (storage → object) - `toJson()` untuk serialize (object → storage) - `copyWith()` untuk immutable updates

✅ **Data Serialization:** - JSON = format text untuk transfer/simpan data - `jsonEncode()` = Object → JSON string - `jsonDecode()` = JSON string → Object

✅ **SharedPreferences:** - Local storage untuk data simple key-value - Persist setelah app ditutup - Cocok untuk settings, preferences, small data - Tidak cocok untuk data besar atau sensitive

✅ **Persistent CRUD:** - Setiap operasi (Create, Update, Delete) harus save ke storage - Load data saat app start di `construct()` - Always handle errors dengan try-catch - Use `async/await` untuk storage operations

---



## Resources

- **Dart JSON Serialization:** <https://dart.dev/guides/json>
  - **SharedPreferences Package:** [https://pub.dev/packages/shared\\_preferences](https://pub.dev/packages/shared_preferences)
  - **Nylo Models:** <https://nylo.dev/docs/6.x/models>
  - **Flutter Data Persistence:** <https://docs.flutter.dev/cookbook/persistence>
- 



## Navigation

[← Sesi 3 - State Management & Local Data](#) | [Sesi 5 - Integrasi Supabase →](#)

---

*Workshop Material - Simple ToDo List with Flutter Nylo | Dokumentasi Sesi 4 - Model & Data Persistence | Terakhir diperbarui: November 21, 2025*